PRD: Multi-Tenant Aggregation Layer — Problem, Goals and Requirements

28 October 2026 · Aroop Ghosh

1. Overview and problem statement

Build a system that shows dashboard numbers (counts, trends, top-10 lists) for many customers quickly, keeps them close to live, and does not slow down the main database.

The setting. Imagine an online store platform like Shopify. Each store owner is a *tenant*. Each tenants' orders, products and customers sit in their respective database. Every company owner who is having multiple stores wants a dashboard: "How many orders today?", "Sales trend this month", "Top 10 products this week".

The problem. With a few thousand records, a simple database query answers this instantly. With millions of records, the same query:

- **Is slow:** it scans lots of data, so the dashboard takes seconds or minutes to load.
- **Hurts everyone:** heavy queries slow down normal reads and writes, so one big store's report makes the app slow for all stores.
- **Forces bad shortcuts:** a nightly job is cheap but shows yesterday's numbers; querying live is fresh but expensive.

Your challenge. Build an aggregation layer that sits between the database and the dashboards. Show that it is **fast**, **fresh**, **correct**, keeps each tenant's data **separate**, and still works when the data grows. You can choose any approach and any tech stack.

2. Goals and non-goals

A good solution answers dashboard queries in under a second, shows new data within a few minutes, and does not make the main database slower.

Goals

1. **Fast:** dashboard queries return in under 1 second.
2. **Fresh:** new orders show up in the numbers within a few minutes (faster is better).
3. **Safe for the database:** dashboard queries do not slow down normal app reads and writes.
4. **Separate tenants:** a tenant only ever sees its own data.
5. **Correct:** the numbers match what a direct database query would return.
6. **Scales:** it still works well when data grows 10 times.

Not required

- A polished dashboard UI (a simple page, Postman or a notebook is fine).
- Login, payments or cloud deployment.
- Machine learning or predictions.

What you will learn: database indexing, pre-computing results, caching, background jobs or streams, and how to measure performance.

3. Functional requirements

Your system must answer the four dashboard queries below for any tenant through a simple API; the bonus items earn extra points.

Must have

ID	Feature	Example
FR-1	**Counts and totals** for a time range	Number of orders and total sales in the last 7 days
FR-2	**Trend over time** (per hour or per day)	Orders per day for the last 30 days
FR-3	**Top-N list**	Top 10 products by sales this week
FR-4	**Breakdown by a category**	Orders by status (placed, shipped, cancelled)
FR-5	**Tenant filter** on every query	Store A can never see Store B's numbers
FR-6	**New data shows up** in the results, including updates (e.g. an order changes from placed to cancelled)	Add an order, then see the count go up
FR-7	**Simple API** (REST is fine) with a short README of the endpoints	GET `/tenants/42/orders/count?from=...&to=...`

Bonus (pick any)

ID	Feature
B-1	Response shows how fresh the data is ("updated 20 seconds ago")
B-2	Rebuild all numbers from scratch if something goes wrong
B-3	Limit how many heavy queries one tenant can run at a time
B-4	Add a new metric by editing a config file, not code
B-5	A simple monitoring page (query time, data lag)

4. Performance targets and setup

Everything should run on a normal laptop with 10 million records across 100 tenants; hit the Good column to qualify and the Great column to stand out.

Data size

- Small test run: **1 million** records, 10 tenants.
- Main run: **10 million** records, 100 tenants (a few tenants are much bigger than the rest, like real life).
- Organisers share a script that generates the data (orders, products, customers).

Targets

What we measure	How	Good	Great
Query speed	Time for each dashboard query at 10M records	Under 1 second	Under 200 ms
Freshness	Time from adding an order to seeing it in the numbers	Under 5 minutes	Under 30 seconds
Database safety	Normal app query time, with and without dashboard load	Less than 25% slower	Less than 10% slower
Correctness	Compare your numbers with a direct database query	Matches on most queries	Matches on all queries
Growth	Query speed at 1M vs 10M records	Less than 5x slower	Less than 2x slower

Suggested tools (not mandatory): MongoDB or PostgreSQL as the main database; Redis for caching; a summary table or collection for pre-computed numbers; cron jobs, change streams or Kafka for updates; ClickHouse or DuckDB if you want a separate analytics store.

Must run easily: judges should be able to start it by following the README (Docker Compose is a plus).